



Java

Project Workbook
Student Edition

Java

Project Workbook

First Edition

LearnKey creates signature multimedia courseware. LearnKey provides expert instruction for popular computer software, technical certifications, and application development with dynamic video-based courseware and effective learning management systems. For a complete list of courses, visit <https://www.learnkey.com>.

All rights reserved. Unauthorized reproduction or distribution is prohibited.

© 2021 LearnKey
www.learnkey.com

Table of Contents

Introduction

Using This Workbook	5
Best Practices Using LearnKey's Online Training	6
Skills Assessment	7
Java Time Tables	8

Domain 1

Fill-in-the-Blanks Domain 1	10
Setting Up Main	11
Signature of Main and Class Instances	12
Command-Line Arguments	13
Print Statements	14
Using the Scanner Class	15
Declaring a Variable	16

Domain 2

Fill-in-the-Blanks Domain 2	18
Data Types and Initialization	19
Primitives vs. Wrapper Object Types	20
String Manipulation	21
String.format Method	22
String Operators	23
Converting a Primitive Data Type to a String	24
Nature of Strings, Initialization, and Null	25
One and Two-Dimensional Arrays	26
Array Lists	27
Implementing Code	28
Arithmetic and Assignment Operators	29

Domain 3

Fill-in-the-Blanks Domain 3	31
Code with Branching Statements	32
Single-Line Statements and Nesting	33
Logical vs. Relational Operators	34
Loops: While, Do While, For, For Each	35
Break, Continue, and Nesting	36
Logical, Relational, and Unary Operators	37

Domain 4

Fill-in-the-Blanks Domain 4	39
Building a Class	40
Constructor Overloading	41
Inheritance and Overriding	42
Class and Data Members	43

Creating Constants and Encapsulation	44
Access Methods	45
Types and Methods	46
Instantiation and Initialization	47
Accessing Data Members and Importing Classes	48

Domain 5

Fill-in-the-Blanks Domain 5	50
Debugging Code and Analyzing Logic Errors	51
Console Exceptions and Stack Trace	52
Try Catch Finally and Exceptions	53
Exception Class Types	54

Appendix

Java Domain 1 Outline	56
Java Domain 2 Outline	57
Java Domain 3 Outline	58
Java Domain 4 Outline	59
Java Domain 5 Outline	60

Using This Workbook

This project workbook contains practice projects and exercises to reinforce the knowledge you have gained through the video portion of the **Java** course. The purpose of this workbook is twofold. First, get you further prepared to pass the Java exam, and second, to get you job-ready skills and increase your employability for jobs needing Java, a very popular object-oriented programming language.

The projects within this workbook follow the order of the video portion of this course. To save your answers in this workbook, you must first download a copy to your computer. You will not be able to save your answers in the web version. You can complete the workbook exercises as you go through each section of the course, complete several of these at the end of each domain, or complete them after viewing the entire course. The key is to go through these projects to strengthen your knowledge of this course.

Each project is based upon a specific video (or videos) in the course and specific test objectives. The materials you will need for this course include:

- The Java Development Toolkit (go to www.java.com and install JDK version 8)
- Visual Studio Code software. You will need to configure Visual Studio Code to work with Java. Use this link: [Getting Started with Java in Visual Studio Code](#)
- The practice files for these projects.

For Teachers

LearnKey is proud to provide extra support to instructors upon request. For your benefit as an instructor, we also provide an instructor support .zip file containing answer keys, completed versions of the workbook project files, and other teacher resources. This .zip file is available within your learning platform's admin portal.

Notes

- Extra teacher notes, when applicable, are in the Project Details box within each exercise.
- Exam objectives are aligned with the course objectives listed in each project, and project file names correspond with these numbers.
- The Finished folder in each domain has reference versions of each project. These can help you grade projects.
- Short answers may vary but should be similar to those provided in this workbook.
- Teachers may consider asking students to add their initials, student ID, or other personal identifiers at the end of each saved project.
- Refer to your course representatives for further support.

We value your feedback about our courses. If you have any questions, comments, or concerns, please let us know by visiting <https://about.learnkey.com>.

Best Practices Using LearnKey's Online Training

LearnKey offers video-based training solutions that are flexible enough to accommodate private students and educational facilities and organizations.

Our course content is presented by top experts in their respective fields and provides clear and comprehensive information. The full line of LearnKey products has been extensively reviewed to meet superior standards of quality. Our course content has also been endorsed by organizations such as Certiport, CompTIA®, Cisco, and Microsoft. However, it is the testimonials given by countless satisfied customers that truly set us apart as leaders in the information training world.

LearnKey experts are highly qualified professionals who offer years of job and project experience in their subjects. Each expert has been certified at the highest level available for their field of expertise. This expertise provides the student with the knowledge necessary to obtain top-level certifications in their chosen field.

Our accomplished instructors have a rich understanding of the content they present. Effective teaching encompasses presenting the basic principles of a subject and understanding and appreciating organization, real-world application, and links to other related disciplines. Each instructor represents the collective wisdom of their field and within our industry.

Our Instructional Technology

Each course is independently created based on the manufacturer's standard objectives for which the course was developed.

We ensure that the subject matter is up-to-date and relevant. We examine the needs of each student and create training that is both interesting and effective. LearnKey training provides auditory, visual, and kinesthetic learning materials to fit diverse learning styles.

Course Training Model

The course training model allows students to proceed through basic training, building upon primary knowledge and concepts to more advanced application and implementation. In this method, students will use the following toolset:

Pre-assessment: The pre-assessment is used to determine the student's prior knowledge of the subject matter. It will also identify a student's strengths and weaknesses, allowing the student to focus on the specific subject matter he/she needs to improve the most. Students should not necessarily expect a passing score on the pre-assessment as it is a test of prior knowledge.

Video training sessions: Each training course is divided into sessions or domains and lessons with topics and subtopics. LearnKey recommends incorporating all available external resources into your training, such as student workbooks, glossaries, course support files, and additional customized instructional material. These resources are located in the folder icon at the top of the page.

Exercise labs: Labs are interactive activities that simulate situations presented in the training videos. Step-by-step instructions and live demonstrations are provided..

Post-assessment: The post-assessment is used to determine the student's knowledge gained from interacting with the training. In taking the post-assessment, students should not consult the training or any other materials. A passing score is 80 percent or higher. If the individual does not pass the post-assessment the first time, LearnKey recommends incorporating external resources, such as the workbook and additional customized instructional material.

Workbook: The workbook has various activities, such as fill-in-the-blank worksheets, short answer questions, practice exam questions, and group and individual projects that allow the student to study and apply concepts presented in the training videos.

Skills Assessment

Instructions: Rate your skills on the following tasks from 1-5 (1 being needs improvement, 5 being excellent).

Skills	1	2	3	4	5
Describe the use of main in a Java application.					
Perform basic input and output using standard packages.					
Evaluate the scope of a variable.					
Declare and use primitive data type variables.					
Construct and evaluate code that manipulates strings.					
Construct and evaluate code that creates, iterates, and manipulates arrays and array lists.					
Construct and evaluate code that performs parsing, casting, and conversion.					
Construct and evaluate arithmetic expressions.					
Construct and evaluate code that uses branching statements.					
Construct and evaluate code that uses loops.					
Construct and evaluate a class definition.					
Declare, implement, and access data members in a class.					
Declare, implement, and access methods.					
Instantiate and use a class object in a program.					
Troubleshoot syntax errors, logic errors, and runtime errors.					
Implement exception handling.					

Java Video Times

Domain 1	Actual Time
Introduction	00:04:21
The Use of Main	00:08:30
Basic Input and Output	00:04:59
Variable Scopes	00:08:44
Total Time	00:26:34

Domain 2	Actual Time
Primitive Type Variables	00:06:30
String Manipulation	00:10:49
Arrays and ArrayLists	00:07:53
Parsing, Casting, and Conversion	00:04:25
Arithmetic and Assignment Operators	00:05:14
Total Time	00:34:51

Domain 3	Actual Time
Branching Statements	00:13:10
Loops	00:15:19
Total Time	00:28:29

Domain 4	Actual Time
Classes	00:12:52
Class Data Members	00:08:26
Methods	00:11:39
Instantiate and Use a Class Object	00:08:18
Total Time	00:41:15

Domain 5	Actual Time
Syntax, Logic, and Runtime Errors	00:09:25
Exception Handling	00:08:55
Course End	00:01:03
Total Time	00:19:23



Java

Domain 1

Fill-in-the-Blanks Domain I

Instructions: While watching Domain I, fill in the missing words according to the information presented by the instructor. [References where answers are found are in brackets.]

The Use of Main

1. Every Java file has to have a as that is the starting point for running a Java file. [Signature of Main]
2. Being that Java is an object-oriented programming language, drive your programs. [Consume a Class Instance]
3. The command compiles a Java file. [Command-Line Arguments]

Basic Input and Output

4. The command runs print statements onto different lines. [Print Statements]
5. The class is one of many built-in classes that needs to be imported before we can use it. [Import and Use the Scanner Class]

Variable Scopes

6. Variables are that store data. [Variables in Blocks]

Setting Up Main

Every Java file needs a method called main, so it is important to know how to set it up when starting your Java code. First, a main method needs to be set up inside of a class, which must match the file name. Once a class is declared, your main method can be created using different keywords. If void is added to your main method, it means that it does not need to return a value. The static keyword indicates that the method can be called without needing an object.

Purpose:

Upon completing this project, you will be able to set up a main method.

Steps for Completion:

1. Open a blank project file in Visual Studio code or your Java developer of choice.
2. Type the following code:

```
class ex11a {  
    public static void main (String args[])  
    {  
        System.out.print("Welcome to Java");  
    }  
}
```
3. Save the file as **ex11a.java**
4. If you are using Visual Studio Code and have installed the Java extensions as described in the How to Set Up Visual Studio Code video, right-click the code window and click Run Code to run the code to view the message, Welcome to Java. Otherwise, run the code in your Java editor to view the message, Welcome to Java.

Project Details

Project File:

N/A

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain I

Topic: The Use of Main

Subtopic: First Java File

Objectives Covered:

I Java Fundamentals

I.I Describe the use of main in a Java application

Signature of Main and Class Instances

Every Java file needs to have a main method, as that is the starting point for running a Java file. Even though a file can have multiple methods, the main method will always run first. Without it, you will not have a runnable Java file. The main method also needs to be declared as a static method, which means that this main method can be run without code to tell it to run, or create an instance of it. The signature for the main method is the area that can hold an array (a group of strings).

Because Java is an object-oriented programming language, objects drive your programs. A class is an object template of sorts, and in this project, you will consume an instance of a class that has been built.

Purpose:

Upon completing this project, you will be able to consume an instance of your own class.

Steps for Completion:

1. Open the project file entitled **ex112.java**
2. On the line below the existing `product1` declaration, declare an instance of the `Product` class, naming it `product2`, setting the type to `Tea`, the name to `Organic Tea`, and the price to `7.99`.
3. On the line below the existing command to print `product1`, add a line to print `product2` on its own line.
4. Save your changes.
5. Run the code to view the new names of the products.

Project Details

Project File:

ex112.java

Estimated Completion Time:

5 minutes

Video Reference:

Domain I

Topic: The Use of Main

Subtopic: Signature of Main;
Consume a Class Instance

Objectives Covered:

I Java Fundamentals

I.1 Describe the use of main in a Java application

I.1.1 Signature of main

I.1.2 How to consume an instance of your own class

Command-Line Arguments

Java code can be run from its developing tool, such as Visual Studio Code. However, it is more likely that this code will be compiled and then run as an app, or as part of an app. One way to run Java code is from the command line. Running a code file is a two-step process from the command line.

In this project, the `String args` signature in the main method declaration has brackets, which means you can have an array of strings. This particular array has two args, or two arguments. The print statement on line 8 takes the two strings, parses them as integers, and then multiplies them. To investigate command-line arguments further, you can type: **javac -help**, which will tell you more ways to run the compiler.

Purpose:

Upon completing this project, you will be able to understand and run command-line arguments. Note that if you are using a tool other than Visual Studio code, the steps for this project will differ.

Steps for Completion:

1. Open the project file entitled **ex113.java**
2. Within the Visual Studio Code tool, click the Terminal tab.
3. At the end of the last line presented in the Terminal tab, type: **javac ex113.java**
4. Press the Enter key to compile the code.
5. At the end of the new line, type: **java ex113** and two numbers to be multiplied. For example, **java ex113 9 8**
6. Press the Enter key to view the correct total of the two multiplied numbers. In our example, the answer should come up as 72.

Project Details

Project File:

ex113.java

Estimated Completion Time:

5 minutes

Video Reference:

Domain I

Topic: The Use of Main

Subtopic: Command-Line Arguments

Objectives Covered:

I Java Fundamentals

I.1 Describe the use of main in a Java application

I.1.3 Command-line arguments

Print Statements

There are a few ways we can print information in the console. One way is to use the command, `System.out.print`; out is short for output. Because Java is a case-sensitive language, `System` has to be capitalized as it is a Java keyword. When it comes to print statements, if you run one print statement after another, everything prints consecutively. To separate print statements onto different lines, you will need to use the command, `println`. To separate text within a print statement, you can add a tab stop using the code, `\t`; or you can add a new line using the code, `\n`.

One tip to remember when learning Java is that you can hover your mouse over some of the keywords you type. Doing this will often bring up tool tips that you can read to learn more about certain keywords and the Java language itself.

Purpose:

Upon completing this project, you will be able to understand and use print statements in Java code.

Steps for Completion:

1. Open the project file entitled **ex121.java**
2. Change the three print statements so that they each print on their own lines.
3. Insert a tab stop after the word, `Small`.
4. Insert a new line after the word, `Large`.
5. Save the file.
6. Run the code. Your output should look like this:

Coffee

Small 1.00

Large

2.00

Project Details

Project File:

ex121.java

Estimated Completion Time:

5 minutes

Video Reference:

Domain I

Topic: Basic Input and Output

Subtopic: Print Statements

Objectives Covered:

I Java Fundamentals

I.2 Perform basic input and output using standard packages

I.2.1 Print statements

Using the Scanner Class

The built-in scanner class handles the inputs. System.in is short for input, the opposite of System.out, which is used for output data. The scanner class is one of many built-in classes that needs to be imported before it can be used. The import.java.util.Scanner command imports the scanner class.

Purpose:

Upon completing this project, you will be able to import and use the scanner class.

Steps for Completion:

1. Open the project file entitled **ex122.java**
2. On line 1 of the file, type the code necessary to enable use of the scanner class.
3. Save the file.
4. Click the Terminal tab.
5. Type: **javac ex122a.java** at the end of the first line and press the Enter key to compile the file.
6. To run the file, type: **java ex122a** at the end of the new line and press the Enter key.
7. Press the Enter key, and the message, "Thank you for visiting us. What is your name?" should pop up. Type your name on the following line.
8. Press the Enter key to test the code. It should welcome you using the name you input.

Project Details

Project File:

ex122.java

Estimated Completion Time:

5 minutes

Video Reference:

Domain I

Topic: Basic Input and Output

Subtopic: Import and Use the Scanner Class

Objectives Covered:

I Java Fundamentals

I.2 Perform basic input and output using standard packages

I.2.2 Import and use the Scanner class

Declaring a Variable

Variables are containers that store data. The intent is that that data will be used later on. It is important to declare variables in the right place. A variable, by default, is only visible within the block of code in which it is declared. As your Java programs grow, you will often have multiple methods within a single class. Some of those methods may need to share variables.

When you go to declare variables, ask yourself the following questions: Do I only need it in this block of code? Do I need this to work for an entire method? How about for an entire class full of methods? Be sure to declare your variables in the correct scope accordingly.

As with any programming language, it is important to understand the regular comment syntax within a Java file. A single line comment starts with two slashes (`//`). A multi-line comment starts (`/*`) and ends (`*/`) with an asterisk and a slash.

The comment syntax also differs when using Javadoc, a documentation generator that takes Java source code and generates documentation for the code in HTML format. The format for a multi-line Javadoc comment includes an extra asterisk at the top line of a comment (`/**`) and a leading asterisk at the beginning of each line of the comment.

Purpose:

Upon completing this project, you will be able to declare a variable within a block, class, and method.

Steps for Completion:

1. Open the project file entitled **ex131a.java**
2. Run the code. You should get an error message about the `salesTax` variable since it is only available in the `if` statement.
3. Move the `print` statement to a place to where the variable can see it.
4. Run the code. The sales tax should come up in the output box.
5. Save your changes.
6. Open the project file entitled **ex131b.java**
7. Run the code as is. It should come up with an error statement.
8. Move the `salesTax` declaration to where both the `if` and `else` statements can see it.
9. Change the `salesTax` declaration so it does not have a value.
10. In the `if` statement, set the `salesTax` to `.047` if the state is `Utah`.
11. Run the code. The sales tax rate should read as `4.7%`.
12. Save your changes.
13. Open the project file entitled **ex131c.java**.
14. Run the code as is. It should come up with an error statement because the `salesTax` variable cannot be found when called through this method.
15. Add the necessary variable declaration and keyword above the `main` method so that the `salesTax` variable is visible throughout the method.
16. Run the code. The sales tax rate returned should be `5.0%`.

Project Details

Project File:

`ex131a.java`, `ex131b.java`, `ex131c.java`

Estimated Completion Time:

10 minutes

Video Reference:

Domain I

Topic: Variable Scopes

Subtopic: Variables in Blocks;
Variables in Methods; Variables in
Classes

Objectives Covered:

- I Java Fundamentals
 - I.3 Evaluate the scope of a variable
 - I.3.1 Declare a variable within a block, class, or method
 - I.4 Comment and document programs
 - I.4.1 Evaluate the syntax of Javadocs
 - I.4.2 Write syntactically correct code comments



Java

Domain 2

Fill-in-the-Blanks Domain 2

Instructions: While watching Domain 2, fill in the missing words according to the information presented by the instructor. [References where answers are found are in brackets.]

Primitive Type Variables

1. A primitive type of data is a type of data. [Data Types and Initialization]
2. There is one instance with decimals to be careful about, and that is losing precision with decimals. [Lost Precision]
3. Wrapper object types, in Java, expand on primitive variables by allowing these variables to be wrapped into a object and treated like objects and used in array lists, collections, sorting, and searching. [Primitives vs. Wrapper Object Types]

String Manipulation

4. The method converts text to uppercase. [String Classes and Literals]
5. The string.format method allows one to avoid having to text. [String.format Methods]
6. The operator determines which character is at a certain spot in a string. [String Operators]
7. The method converts a primitive data type, like an integer, to a string. [Primitive Data Type to String Conversion]
8. If a variable is not sitting in , it can't be intercepted. [Immutable Nature of Strings]
9. Null does not mean zero. Null means . [Initialization, Null]

Arrays and ArrayLists

10. Arrays are -based. [One-Dimensional Arrays]
11. An array list allows us to declare the array and then or items at will. [ArrayLists and Adding Elements]

Parsing, Casting, and Conversion

12. A double data type, multiplied by an integer, results in a data type. [Casting and Conversion]

Arithmetic and Assignment Operators

13. A remainder after a division is a . [Arithmetic Operators]

Data Types and Initialization

In Java, there are eight primitive types of data. Primitive means predefined, so a range is predefined for each variable type. It is like having different sized boxes that can store different types of items. When declaring a variable, you should declare your type, a name for the variable, and then its value. This is called initializing the variable when it is the first time the variable has been used in a particular code. Remember that variable names can have letters, numbers, underscores, and currency symbols, but they cannot start with a number.

The data type, `char`, defines a single character or a positive number up to 65535 that represents a character. If a letter is used, it needs to be in single quotes. `int` stands for integer, as in a 32-bit whole number, which means it can go up to about 2.1 billion. `Double` is a 64-bit decimal number, and `short` is a 16-bit whole number. `Long` is a 64-bit whole number, and it needs a `l` after the number or Java will think it is an integer. `Float` is a 32-bit decimal, and it needs an `f` after the number or Java will read it as a double. `Boolean` is a simply a variable that will store true or false. You will see these data types in the project.

Camel casing is a suggested naming convention in Java; it simply means to have the first word in lowercase and then subsequent words start with a capital letter.

You will need to be careful about losing precision with floating point decimals. For very large numbers with decimals, use the double data type instead of float to avoid problems, as the double data type handles large numbers far better than does the float data type.

Purpose:

Upon completing this project, you will be able to add comments to code and identify when precision is lost.

Steps for Completion:

1. Open the project file entitled **ex211.java**, and notice how the variables are declared.
2. After the character variable, add the comment, add a comment indicating that the variable is a single character.
3. Save your changes.
4. Open the project file entitled **ex212.java**.
5. Change the cost per ounce within the floating data type from `.49` to a number between 500,000 and 600,000 (this must include a decimal). For example, `555888.33f`.
6. Save the file.
7. Run the code. You will notice that there is a floating point result that is slightly inaccurate.

Project Details

Project Files:

ex211.java, ex212.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Primitive Type Variables

Subtopic: Data Types and Initialization; Lost Precision

Objectives Covered:

- 2 Data Types, Variables, and Expressions
 - 2.1 Declare and use primitive data type variables
 - 2.1.1 Data types, including byte, char, int, double, short, long, float, Boolean
 - 2.1.2 Identify when precision is lost
 - 2.1.3 Initialization

Primitives vs. Wrapper Object Types

Primitive variables have defined ranges. Wrapper object types, in Java, expand on primitive variables by allowing them to be wrapped into a collection object and treated like objects and used in array lists, collections, sorting, and searching.

Purpose:

Upon completing this project, you will be able to identify the differences between primitive and wrapper object types.

Steps for Completion:

1. Open the project file entitled **ex214.java**
2. On line 5, declare a primitive integer setting the variable qty to 20.
3. On line 6, declare a wrapper object for an integer named reorder with the quantity set to 10.
4. On line 8, declare a wrapper object for a Boolean named needsReorder set to false.
5. On line 9, declare a primitive Boolean variable named mustReorder and set its value to the value of needsReorder.
6. Save the file.
7. Run the code. The quantity is higher than the reorder number, so it does not need to be reordered.

Project Details

Project File:

ex214.java

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 2

Topic: Primitive Type Variables

Subtopic: Primitives vs. Wrapper Object Types

Objectives Covered:

2 Data Types, Variables, and Expressions

2.1 Declare and use primitive data type variables

2.1.4 How primitives differ from wrapper object types such as Integer and Boolean

String Manipulation

Unique to Java, the word string starts with a capital S. The string container is not limited in size, so it is not considered primitive; it is an object type. Object types start with a capital letter. In the following file, the following tasks have been done with the displayed strings: compare, concatenate, change case, show length, and see if one of these is empty.

Purpose:

Upon completing this project, you will be able to understand different string tasks.

Steps for Completion:

1. Open the project file entitled **ex221.java**
2. On line 8, add a print statement testing to see if product1 is not the same as product2.
3. On line 9, add a print statement concatenating the text, Our house specials today are, with product1, the word and, and product2.
4. On line 10, add a statement similar to line 9 (you can copy and paste) but with the products converted to uppercase.
5. On line 11, add a print statement to print the length of product1.
6. On line 12, add a print statement to check to see if the product1 value is empty.
7. Save your changes.
8. Run the code. You should see the following output:

true

Our house specials today are House Blend and Sumatra.

Our house specials today are HOUSE BLEND and SUMATRA.

11

false

Project Details

Project File:

ex221.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: String Manipulation

Subtopic: String Classes and Literals

Objectives Covered:

2 Data Types, Variables, and Expressions

2.2 Construct and evaluate code that manipulates strings

2.2.1 String class and string literals, comparisons, concatenation, case, and length

String.format Method

The String.format method allows you to take a string variable and format it, along with other text. This can save you from having to write a concatenating statement, and it allows some flexibility with formatting text. You can use string manipulation to control not only the output of a string, but the length it takes up as well. As an example, the String.format method with the text `[%15s]` before a string tells the string to print in between two vertical pipes and occupy 15 characters.

Purpose:

Upon completing this project, you will be able to use the String.format method in Java code.

Steps for Completion:

1. Open the project file entitled **ex222.java**
2. Run the code. The products should print with vertical pipes on either side.
3. On line 9, add the code necessary to display product2 in between two vertical pipes and occupy 20 total characters.
4. Save your changes.
5. Run the code. Your output should look like this:

```
|House Blend|
|          Sumatra|
```

Project Details

Project File:

ex222.java

Estimated Completion Time:

5 minutes

Video Reference:

Domain 2

Topic: String Manipulation

Subtopic: String.format Methods

Objectives Covered:

2 Data Types, Variables, and Expressions

2.2 Construct and evaluate code that manipulates strings

2.2.2 String.format methods

String Operators

String operators are built-in string methods. A few of the operators that will be used in this project include the trim method, substring method, and the charAt method. The trim method trims extra spaces from a string of text, the substring method extracts part of a string, and the charAt method determines which character is at a certain spot in a string. Remember that Java is a zero-based language, meaning that for characters and arrays, the first number is 0, not 1; this is important to know when using the charAt method.

Purpose:

Upon completing this project, you will be able to apply different operators to strings.

Steps for Completion:

1. Open the project file entitled **ex223.java**
2. Notice the strings of data on lines 5 and 6. This is simulating an import from a text file to where columns of data have spaces in between them.
3. On line 9, add the code needed to print out just the words, White Tea, from the tea2 variable.
4. On line 10, add the code needed to print the fourth character in the tea1 variable.
5. Save your changes.
6. Run the code. You should see the following:

```
0001  Green Tea    7.99
      White Tea
      |
```

Project Details

Project File:

ex223.java

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 2

Topic: String Manipulation

Subtopic: String Operators

Objectives Covered:

2 Data Types, Variables, and Expressions

2.2 Construct and evaluate code that manipulates strings

2.2.3 String operators

Converting a Primitive Data Type to a String

Sometimes you will need to take a primitive data type, such as an integer, double, or Boolean, and convert it to a string. One reason for this is that an output may need to be in an all-string format. When using the `toString` method, it is important to specify what you are converting to a string, and that whatever it is is capitalized. This statement can be run over multiple lines since the semicolon is what ends the statement.

`String.valueOf` is a conversion method which can be used on any primitive data type and is often used when you do not want to have to dig through to see what the original data type was in the first place.

Purpose:

Upon completing this project, you will be able to convert primitive data to strings.

Steps for Completion:

1. Open the project file entitled **ex224.java**
2. Notice the output `toString` on line 9. This converts the quantity to a string.
3. Copy the code on line 9 and paste it on the next line.
4. Change the code on line 10 to convert the `qty` variable to a string using the `valueOf` method.
5. Save your changes.
6. Run the code. You should see the same output twice, as seen here:

Tracy, thank you for your order of 5 cans of Mint Cocoa.

Tracy, thank you for your order of 5 cans of Mint Cocoa.

Project Details

Project File:

ex224.java

Estimated Completion Time:

5 minutes

Video Reference:

Domain 2

Topic: String Manipulation

Subtopic: Primitive Data Type to String Conversion

Objectives Covered:

2 Data Types, Variables, and Expressions

2.2 Construct and evaluate code that manipulates strings

Nature of Strings, Initialization, and Null

Strings variables are immutable, which means they cannot be changed. Sometimes when you think the string variable is changed, it is actually done away with in memory and then reallocated to memory. This is done for a couple of reasons. The first is security. If a variable is not sitting in memory, it cannot be intercepted. Secondly, if it is not sitting in memory, it is not susceptible to memory leaks.

When it comes to null, null does not mean zero. Null means empty. If you want a variable to have nothing in it, you should set it to null.

Purpose:

Upon completing this project, you will be able to understand and recognize the immutable nature of strings, as well as the use of initialization and null.

Steps for Completion:

1. Open the project file entitled **ex225.java**
2. Change the declaration of the status variable to be declared but not initialized.
3. Save your changes.
4. Run the code. The status should print with the value, Silver.
5. Open the project file entitled **ex226.java**
6. Set the string variable named status to a value of, unknown. This will initialize the variable because it now has a set value.
7. Save the file and run the code. Because the quantity is set to 70, the output returns Silver.
8. Change the total quantity to 40.
9. Save the file and run the code. The output should be null because the total is less than 50.

Project Details

Project Files:

ex225.java; ex226.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: String Manipulation

Subtopic: Immutable Nature of Strings; Initialization, Null

Objectives Covered:

2 Data Types, Variables, and Expressions

2.2 Construct and evaluate code that manipulates strings

2.2.4 The immutable nature of strings

2.2.5 Initialization

2.2.6 Null

One and Two-Dimensional Arrays

Arrays are containers that have multiple compartments to store items. Those items can be referred to by their index number. An array will often use square brackets with a number inside to indicate the quantity of items it holds. Arrays are zero-based, meaning the first item in the array has an index of zero. This means you want the last index number to be one less than the number you put in for the size, so that you do not end up with an error because the index number exceeds the array size. As arrays are built, you can access a single array element, or the entire list, through accessing its index number.

A one-dimensional array is kind of like having a box with a single row of compartments, each with a space for an item. If you need to have two rows of compartments, with each row having some meaning for hosting two parts, then you can use a two-dimensional array.

Purpose:

Upon completing this project, you will be able to access a single array element and a two-dimensional array element.

Steps for Completion:

1. Open the project file entitled **ex23 1a.java**
2. After the print statement to print a cocoa brand, declare a for loop for the string variable called cocoa, point it at the cocoaBrands array, and print each cocoa brand.
3. Save the file and run the code. You should see element 1, Dark Chocolate, and then all of the types of cocoa iterated through. Close the project file.
4. Open the project file entitled **ex23 1b.java**
5. On lines 10 through 12, following the cocoa products, add three coffee products to the array list, using the products[1] array slot. The products should be: Coffee - House, Coffee - Decaf, and Coffee - French Vanilla.
6. Add a line to print the product at array slot [1] [1].
7. Save the file.
8. Run the code. The decaf coffee product should be the output.
9. Save the file as ex23 1bb.java, and run the code. The product from index 1, 1 should output the decaf coffee. Close the project file.
10. Open the project file entitled **ex23 1c.java**
11. On line 5, declare a new string array called cocoaBrands with the products, Regular, Dark Chocolate, null, and Sugar Free.
12. Save the file.
13. Run the code. You should see all of the products in the output.

Project Details

Project Files:

ex23 1a.java, ex23 1b.java, ex23 1c.java

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 2

Topic: Arrays and ArrayLists

Subtopic: One-Dimensional Arrays;
Two-Dimensional Arrays; Other
Array Creation Methods

Objectives Covered:

2 Data Types, Variables, and Expressions

2.3 Construct and evaluate code that creates, iterates, and manipulates arrays and array lists

2.3.1 One- and two-dimensional arrays, including initialization, null, size, iterating elements, accessing elements

LearnKey ArrayLists

Because you cannot add to the size of the array once it is declared, you can use an ArrayList to declare the array and then add or remove items at will. An ArrayList uses angle brackets for data, similar to many programming languages which use angle brackets for lists. One way to display items is through the use of an iterator, which allows you to traverse through an ArrayList. You will also need to import some Java utilities to use the ArrayList and Iterator types. The remove method removes an item from an ArrayList.

Purpose:

Upon completing this project, you will be able to add and remove elements from an ArrayList.

Steps for Completion:

1. Open the project file entitled **ex232.java**
2. Notice the new ArrayList on line 7, the additions made to cups on lines 9 through 12 using the add method, the get method used on line 14, the iterator on line 16, the while loop on line 17, and the code, import java.util.* code on line 1.
3. Run the code as it is. You should see the item with the index of 1, the black cup, and then the entire array.
4. Add a blank line below the end of the while statement and curly brackets.
5. On the new blank line, add code to remove the maroon cup from the array.
6. Copy the iteration statement, without the Iterator keyword as it does not need to be declared again, and paste it below the existing code.
7. Copy the while statement on lines 17 through 20 and paste it after the iteration statement.
8. Save the file.
9. Run the code. You should see item 1, the four cups, and then the remaining three cups after the maroon cup has been removed.

Project Details

Project File:

ex232.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 2

Topic: Arrays and ArrayLists

Subtopic: ArrayLists and Adding Elements; ArrayLists and Removing Elements

Objectives Covered:

2 Data Types, Variables, and Expressions

2.3 Construct and evaluate code that creates, iterates, and manipulates arrays and array lists

2.3.2 ArrayLists, including adding and removing elements, traversing the list

Implementing Code

When converting data types in Java, keep in mind that you must determine whether you are converting one primitive data type to another, or a primitive data type to an object type.

Purpose:

Upon completing this project, you will be able to convert numbers to strings, and strings to numbers.

Steps for Completion:

1. Open the project file entitled **ex241a.java**
2. Within the main method, and below the two existing variables, declare a variable named, `bagSizeWhole`, that is an integer conversion from the `bagSize` variable. Then declare a variable named, `bagSizeString`, that is a string conversion from the `bagSize` variable.
3. Save the file.
4. Run the code. You should see the following output, with the integer variable cutting off the decimal.

12.9

12

12.9

51.6

5. Open the project file entitled **ex241b.java**
6. Run the code. Why are you getting an error message?

7. Comment out the two lines causing the error message.
8. Save the changes and run the code. You should see a value of 11.8.

Project Details

Project Files:

ex241a.java, ex241b.java

Estimated Completion

Time:

5-10 minutes

Video Reference:

Domain 2

Topic: Parsing, Casting, and Conversion

Subtopic: Casting and Conversion; Parse Strings to Numbers

Objectives Covered:

- 2 Data Types, Variables, and Expressions
 - 2.4 Construct and evaluate code that performs parsing, casting, and conversion
 - 2.4.1 Cast between primitive data types
 - 2.4.2 Convert primitive types to equivalent object types
 - 2.4.3 Parse strings to numbers
 - 2.4.4 Convert primitive data types to strings

Arithmetic and Assignment Operators

The final topic in this domain focuses on operators: arithmetic operators, compound assignment operators, and operator precedence. Arithmetic operators include the following: addition, subtraction, multiplication, division, and modulus, which is the remainder after division.

When it comes to assignment operators, the most common one is the equal sign. The compound assignment operators include `+=`, `*=`, `-=`, and `/=`. These shortcut operators actually assign a new value to `x` with each of their equations.

The order of operator precedence, or order of operations, you need to know is as follows: parentheses, exponents, multiplication, division, addition, and subtraction. A good way to remember this is by taking the first letter of each word and coming up with an expression, such as "Please Excuse My Dear Aunt Sally."

Purpose:

Upon completing this project, you will be able to identify and understand different types of operators in Java.

Steps for Completion:

1. Open the project file entitled **ex251a.java**
2. Using the comments as your guide, fill out, in the parentheses for each print statement, the correct code to calculate `x` and `y` (add, subtract, multiply, divide, and modulus).
3. Save your changes.
4. Run the code. You should see results of 22, 8, 105, 2, and 1.
5. Open the project file entitled **ex251b.java**
6. In front of each comment, add a calculation for `x` and `y` using the compound assignment operator for each calculation (add, subtract, multiply, and divide).
7. Save your changes.
8. Run the code. You should see results of 22, 8, 105, and 2.
9. Open the project file entitled **ex251c.java**
10. Run the code. You should see a value of 36.
11. Copy the print statement on line 9 and paste it on the line below.
12. Manipulate the equation to get an answer of 66 instead of 36.
13. Save your changes.
14. Run your code. You should see output values of 36 and 66.
15. Close your open files, saving changes when prompted.

Project Details

Project Files:

ex251a.java, ex251b.java, ex251c.java

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 2

Topic: Arithmetic and Assignment Operators

Subtopic: Arithmetic Operators; Assignment Operators; Operator Precedence

Objectives Covered:

- 2 Data Types, Variables, and Expressions
- 2.5 Construct and evaluate arithmetic expressions
 - 2.5.1 Arithmetic operators, assignment, compound assignment operators, operator precedence



Java

Domain 3

Fill-in-the-Blanks Domain 3

Instructions: While watching Domain 3, fill in the missing words according to the information presented by the instructor. [References where answers are found are in brackets.]

Branching Statements

1. For conditions to check, consider using a switch statement instead of using several if statements. [Switch Statements]
2. In a single-line if statement, a condition, followed by a , some text, a colon, and some more text indicates a condition, what to do if the condition is true, and what to do if the condition is false. [Single-Line vs. Block Statements]
3. For if statements, nesting is the act of putting one statement inside of another. [Nesting Branch Statements]
4. A && indicates an while a || indicates an . [Logical Operators]

Loops

5. In a do/while loop, the statements inside the loop will run at least . [Do While Loop]
6. A continue statement skips code in an of a loop. [Continue]
7. In the context of a loop, nesting is the act of putting a loop of a loop. [Nesting Loops]
8. If a ++ occurs after a variable, the variable itself will first and then be increased by one. [Unary Operators]

Code with Branching Statements

There are a few types of conditional statements you can use in Java code, namely if, else, else if, and switch. An if statement runs in the case if that something happens, then something will definitely occur. The else acts on the if part being false. In the case that the if part of an if-else code block is false, then the else code runs. Sometimes, there is more than one if condition to evaluate in code. You cannot have two ifs before an else, but you can have an else if statement. In a switch statement, if an amount is set to default, that is a reference to all amounts other than any others that have specifically been declared. Unlike an if statement, which is done after the first match, a switch statement will assume that after a match, all of the declared actions should take place. To remedy this, you will need break statements after each case and action. Break means break out of this code.

Purpose:

Upon completing this project, you will be able to understand and use if, else, else if, and switch statements effectively in code.

Steps for Completion:

1. Open the project file entitled **ex311a.java**
2. Replace the comment with a code declaration to check to see if the amount is at least two.
3. Save the changes.
4. Run the code. You should see a message for a free coaster.
5. Change the amount declaration on line 5 to **1**.
6. Save and run the code again. This time, the message should not display because the condition was not met. Close the file.
7. Open the project file entitled **ex311b.java**
8. Under the if statement add a statement that will print the text, Thank you for your order, if the amount is not greater than two.
9. Save and run the code. You should see a message thanking the customer for the order.
10. Open the project file entitled **ex311c.java**
11. In between the if and else statment, add the code necessary to print the message, Take \$1 off of your next order, if the amount equals two.
12. Save and run the code. You should still see the message, Thank you for your order.
13. Change the amount in the variable declaration to **2**.
14. Save and run the code again. This time, you should see the message, Take \$1 off of your next order.
15. Open the project file entitled **ex311d.java**
16. Run the code as it is with the amount set to 1. The message from the default switch should appear in the output.
17. Change the amount on line 5 to **3**.
18. Save your changes.
19. Run the code. You should see three print messages, indicating that there is a problem with the code.
20. Add the code necessary to stop the switch statement when a match is reached.
21. Save and run the code again. You should only see the message starting with the words, Three bags?

Project Details

Project Files:

ex311a.java, ex311b.java, ex311c.java, ex311d.java

Estimated Completion Time:

15 minutes

Video Reference:

Domain 3

Topic: Branching Statements

Subtopic: If Statements; Else Statements; Else If Statements; Switch Statements

Objectives Covered:

3 Flow Control Implementation

3.1 Construct and evaluate code that uses branching statements

3.1.1 If, else, else if, switch

Single-Line Statements and Nesting

If statements are primarily in blocks of code, as seen in previous projects. However, most of those statements can be run in a single line of code. For example, a single-line if statement can be used whenever you have a single if/else situation. Nesting, in the context of conditional statements, is the act of putting one conditional statement inside of another.

Purpose:

Upon completing this project, you will be able to create a single-line if statement, and understand nesting.

Steps for Completion:

1. Open the project file entitled **ex312.java**
2. Replace the comment with the conditional code necessary to finish the single-line if statement that has started. If the condition is true, the message should be, Enjoy a free coaster with your order. If the condition is false, the message should be, Thank you for your order.
3. Save your changes.
4. Run the code. The thank you message should be displayed in the Output window.
5. Open the project file entitled **ex313.java**
6. Replace the comment for adding discount logic with a block of code that will assign .10 to the discount variable if the bagQty is at least 10 and .05 to the discount variable in all other circumstances.
7. Save your changes.
8. Run the code. As a result of having a gold customer who purchased eight bags, you should see the following message:
Congratulations. Your Gold status gets you a 5.0% discount.

Project Details

Project Files:

ex312.java, ex313.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Branching Statements

Subtopic: Single-Line vs. Block Statements; Nesting Branch Statements

Objectives Covered:

3 Flow Control Implementation

3.1 Construct and evaluate code that uses branching statements

3.1.2 Single-line vs. block

3.1.3 Nesting

Logical vs. Relational Operators

This project focuses on two logical operators: and, and or. When it comes to relational operators, there are three basic categories: equal to, greater than or less than, and greater than or equal to or less than or equal to.

Purpose:

Upon completing this project, you will be able to understand and use different types of logical and relational operators.

Steps for Completion:

1. Open the project file entitled **ex314a.java**
2. Below the three string variables, declare two Boolean variables named, bothDarks, and oneDark.
3. Replace the code in the if and else statements to just set the bothDarks variable to be true or false based on both the frRoast and italRoast variables being set to the value, in.
4. Set the oneDark variable to be true or false based on either the frRoast or italRoast variables being set to the value, in.
5. Save your changes.
6. Run the code. The first output should be false and the second one, true.
7. Open the project file entitled **ex314b.java**
8. To the right of each print line statement, add a comment to either indicate whether you think the statement will be true or false.
9. Save your changes.
10. Run the code. Compare your guesses with the outputs.
11. Close your open files, saving changes if prompted.

Project Details

Project File:

ex314a.java, ex314b.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Branching Statements

Subtopic: Logical Operators;
Relational Operators

Objectives Covered:

3 Flow Control Implementation

3.1 Construct and evaluate code that
uses branching statements

3.1.4 Logical and relational operators

Loops: While, Do While, For, For Each

There are four types of loops that are covered in this project, each with a specific purpose. They are while loops, do while loops, for loops, and for each loops. A while statement has two main pieces to it: a condition, and then what to do while the condition is true. You will need to make sure that your loops are not infinite, otherwise your code will use up all your memory and eventually crash. If you want to run a loop at least once, you will want to use a do while loop. A for loop is set for a certain number of times, so it is not as much of a conditional loop as the while loop because the number of times is set within the loop. The for each loop loops through each item in a group or collection. The difference between this and the for loop is that you do not need to define the number of times a loop will loop; the size of the group determines that. It is important to keep in mind which output you are trying to generate when choosing which type of loop to use.

Purpose:

Upon completing this project, you will be able to understand and utilize while, do while, for, and for each loops in Java code.

Steps for Completion:

1. Open the project file entitled **ex321a.java**
2. Replace the comment in the code with the statement that will run the bracketed code so long as orderCount is less than totalOrders.
3. Save your changes.
4. Run the code. Three messages should display in the output.
5. Open the project file entitled **ex321b.java**
6. Run the code. As the orderCount equals totalOrders, the code in the while loop never runs.
7. Adjust the code so that the while loop is guaranteed to run at least once.
8. Save your changes.
9. Run the code. You should see the following:
An order will be sent to you in 30 days.
10. Open the project file entitled **ex321c.java**
11. Replace the existing comment with a for loop that will print out each cocoa brand.
12. Save the changes and run the code. Each of the items in the cocoaBrands array list should appear in the output once.
13. Open the project file entitled **ex321d.java**
14. Replace the existing comment with a for each loop that will print out each cocoa brand.
15. Save the changes and run the code. Each of the items in the cocoaBrands array list should appear in the output once.
16. Close all open files, saving changes if prompted.

Project Details

Project Files:

ex321a.java, ex321b.java, ex321c.java, ex321d.java

Estimated Completion Time:

15 minutes

Video Reference:

Domain 3

Topic: Loops

Subtopic: While Loop; Do While Loop; For Loop; For Each Loop

Objectives Covered:

3 Flow Control Implementation

3.2 Construct and evaluate code that uses loops

3.2.1 While, for, for each, do while

Break, Continue, and Nesting

A break occurs when you are running through any kind of a loop and you then want to exit that loop. While the break statement ends a loop, sometimes you may want to just skip over an iteration of a loop using a continue statement. Nesting, in the context of loops, is the act of putting a loop inside of a loop.

Purpose:

Upon completing this project, you will be able to understand and use break and continue statements, as well as nesting.

Steps for Completion:

1. Open the project file entitled **ex322a.java**
2. Use an if statement to make the loop break if it runs into a product with a null value.
3. Save your changes.
4. Run the code. Only the regular and dark chocolate products should display.
5. Open the project file entitled **ex322b.java**
6. Adjust the code so that the for loop will skip over any products with a value of null.
7. Save your changes.
8. Run the code. All three products should display but there should be no null messages.
9. Open the project file entitled **ex323.java**
10. Replace the commented code with a for loop that will loop through the cocoa brands.
11. Save your changes.
12. Run the code. The output should look like this:

U.S.-Dark Chocolate

U.S.-Mint Chocolate

U.S.-Sugar free

International-Regular

International-Dark Chocolate

International-Mint Chocolate

Project Details

Project Files:

ex322a.java, ex322b.java, ex323.java

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 3

Topic: Loops

Subtopic: Break; Continue; Nesting

Loops

Objectives Covered:

3 Flow Control Implementation

3.2 Construct and evaluate code that uses loops

3.2.2 Break and Continue

3.2.3 Nesting

Logical, Relational, and Unary Operators

Three major logical operators are and, or, and not equal to. The double ampersand represents an and, as in this *and* this must be true. A double vertical pipe is used to represent an or, and an exclamation and equal sign is used for a not equal to operator. A unary operator either increases or decreases a variable by one. The symbol ++ is the unary part of the code. If the ++ occurs after the variable, the variable itself will print first, and then it will be increased by one. It is important to pay close attention to where the unary operator is in relation to the variable because it can make a difference.

Purpose:

Upon completing this project, you will be able to understand and use logical, relational, and unary operators.

Steps for Completion:

1. Open the project file entitled **ex324a.java**
2. Surround the continue statement with an if statement that checks to see if the territories array value is International and the cocoa brands array value is sugar-free.
3. Save your changes.
4. Run the code. You should not see a print out for international, sugar-free cocoa.
5. Open the project file entitled **ex324b.java**
6. Replace the three comments with three print statements, each fulfilling the request of each comment.
7. Save your changes.
8. Run the code. You should see values of 5, 7, and 10.
9. Close all open files, saving changes if prompted.
10. Save the file, and run the code. The numbers 7 and 5 should appear in the output, representing y. Close the file.

Project Details

Project File:

ex324a.java, ex324b.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 3

Topic: Loops

Subtopic: Logical and Relational Operators; Unary Operators

Objectives Covered:

3 Flow Control Implementation

3.2 Construct and evaluate code that uses loops

3.2.4 Logical, relational, and unary operators



Java

Domain 4

Fill-in-the-Blanks Domain 4

Instructions: While watching Domain 4, fill in the missing words according to the information presented by the instructor. [References where answers are found are in brackets.]

Classes

1. A constructor is a [] with the same name as the [] it is using to create an object. [Constructors]
2. Constructor overloads allow [] to be built in multiple ways. [Constructor Overloads]
3. In a Java file, the initial class name and the [] name need to match. [Classes per Java File]
4. The This keyword can be used as a shortcut within methods and []. [The This Keyword]
5. An override is a change to a default method from a [] class. [Overriding]

Class Data Members

6. A [] property can be used in its class plus any inherited class. [Private, Public, Protected]
7. Data members that are not the same every time should be [] data members. [Instance and Static Data Members]
8. The Static Final keywords should be used on a variable that should not be []. [Use Static Final to Create Constants]
9. Encapsulation is the act of [] information. [Encapsulation]

Methods

10. Methods perform [], often in the form of calculations. [Private, Public, Protected Methods]
11. When using parameters in methods, parameters are located inside of []. [Method Parameters]
12. A void method does not return a []. [Return Type and Value, Voids]
13. An instance method creates an instance of a [], while a static method is []. [Instance and Static Methods]
14. Overloading a method involves creating a method with the same [] as the method to overload. [Overloading]

Instantiate and Use a Class Object

15. Instantiating a class involves creating a new [] of a class. [Instatiation and Initialization]
16. Nulls can be set as a [] value for a data member. [Null Use]
17. One way to modify a data member is to change it in its [], meaning its original location. [Access and Modify Data Members]
18. When importing a class, an import statement is not needed so long as the class is not [] and is in the same [] in the file in which it is being used. [Import Packages and Classes]

Building a Class

A class is the base definition for an object, including which characteristics an object will have. In this project, you will build a class for a product, which will have a product ID, type, product name, and price. Each property within the product class will have a data type, which helps to shape the product objects that will be created later. Also, one Java naming convention for classes is to have a capital first letter for each word. Once a class is built, you will want to put one or more constructors in place to build objects from your classes. A constructor is a method with the same name as the class it is using to create an object.

Purpose:

Upon completing this project, you will be able to build a class and a constructor.

Steps for Completion:

1. Open the project file entitled **ex411.java**
2. Run the code as it is. A run-time error should occur, stating that a non-static variable cannot be referenced from a static context.
3. Make the Product class on line 10 static.
4. Save your changes.
5. Run the code. You should see the following:

Our first tea is Green Tea.

Project Details

Project File:

ex411.java

Estimated Completion Time:

5 minutes

Video Reference:

Domain 4

Topic: Classes

Subtopics: Build a Class;
Constructors

Objectives Covered:

4 Object-Oriented Programming

4.1 Construct and evaluate class definitions

4.1.1 Constructors

Constructor Overloading

When building classes and objects in Java, you have multiple ways to build an object by building more constructors. This concept is known as constructor overloading.

In Java, there is a misnomer that there is only one class allowed per .java file. This is simply not the case; what you need to know is that the initial class name and file name need to match. In this project, a class is made public. When it comes to classes, you can only have one public, non-static class per Java file because the main method, which is the starting point for what runs, is static, so anything in the code needs to come from a static class.

The keyword, this, is versatile in many programming languages. In Java, it can be used as a shortcut within methods and constructors.

Purpose:

Upon completing this project, you will be able to use constructor overloading, classes, and the keyword, this, successfully.

Steps for Completion:

1. Open the project file entitled **ex412.java**
2. In the second product constructor, replace the comment with three variables, setting productIdValue to an integer, typeValue to a string, and productNameValue to a string.
3. Within the second constructor, replace the price comment with code to set a default price of 9.99.
4. Save your changes.
5. Run the code. You should see the following:
Our first tea is Green Tea.
The House Blend brand sells for 9.99.
6. Open the project file entitled **ex413.java**
7. Run the code.
8. Fix the error that appears in the output window.
9. Save the code and run it again. You should see messages for the two products.
10. Open the project file entitled **ex414.java**
11. Below the initial product declaration, declare a second product using no parameters and the this keyword to set the productIdValue to 3, the typeValue to Coffee, the productNameValue to Decaf, and the price to 9.99.
12. Save your changes.
13. Run the code. You should see the following:
Our first tea is Green Tea.
The Decaf brand sells for 9.99.
14. Close your open files, saving changes if prompted.

Project Details

Project Files:

ex412.java, ex413.java, ex414.java

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 4

Topic: Classes

Subtopics: Constructor Overloads;
Classes per Java File; The This
Keyword

Objectives Covered:

- 4 Object-Oriented Programming
 - 4.1 Construct and evaluate class definitions
 - 4.1.2 Constructor overloading
 - 4.1.3 One class per .java file
 - 4.1.4 This keyword

Inheritance and Overriding

Classes in Java can have unique variations, which enables them to inherit properties from other classes, if need be. You can also build methods into your classes. An override within a class is a change to a default method used in a parent class (a class you are inheriting from).

Purpose:

Upon completing this project, you will be able to understand and create an inheritance and an override.

Steps for Completion:

1. Open the project file entitled **ex4I5a.java**
2. Replace the comment on line 34 with a statement that declares a class named, GourmetCoffee, that extends the product class.
3. Save your changes.
4. Run the code. You should see the following:
Our first tea is Green Tea.
We have a new Chocolate coffee.
5. Open the project file entitled **ex4I5b.java**
6. Replace the comment on line 52 with a method declaration for a static integer named, initStock, that returns a value of 20.
7. Replace the comment on line 39 with a block of code that will return 10 for a value.
8. Save your changes.
9. Run the code. You should see the following:
Our first tea is Green Tea.
We have a new Chocolate coffee with 10 in stock.
10. Close your open files, saving changes if prompted.

Project Details

Project Files:

ex4I5a.java, ex4I5b.java

Estimated Completion

Time:

10 minutes

Video Reference:

Domain 4

Topic: Classes

Subtopic: Inheritance; Overriding

Objectives Covered:

4 Object-Oriented Programming

4.1 Construct and evaluate class definitions

4.1.5 Inheritance and overriding

Class and Data Members

The term, properties, is somewhat synonymous with members of a class. When using Java, you need to know the scope of your class members, whether they need to be made public, protected, or private. A public class member can be used anywhere within an app. A protected class member can be used in its own class and any class that inherits from it. A private class member can only be used in its class.

There are two types of data members: instance data members, and static data members. From a technical standpoint, an instance member creates a memory space every time it is used. A static data member, on the other hand, only uses one memory space no matter how many times it is called. For data members that are not the same every time, use an instance data member. If they are going to be the same every time, use a static member.

Purpose:

Upon completing this project, you will be able to identify and utilize different types of class and data members.

Steps for Completion:

1. Open the project file entitled **Tea1.java**
2. Run the code as is. An error should occur because the salePrice is a private variable, so it cannot be used on gourmet tea.
3. Fix the code causing the error.
4. Save your changes.
5. Run the code again. You should see the following:
Our first tea is Mint Green sells for 9.99 and is on sale now for 6.99
We have a new gourmet tea called Super Lotus for 12.99.
6. Open the project file entitled **Tea2.java**
7. Replace the comment on line 38 with a variable named gourmetSalePrice set to 10.99. Make sure to use the correct category and data type for the variable.
8. Save your changes.
9. Run the code. You should see the following:
Our first tea is Mint Green sells for 9.99 and is on sale now for 6.99
We have a new gourmet tea called Super Lotus on sale for 10.99.

Project Details

Project Files:

Tea1.java, Tea2.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 4

Topic: Class Data Members

Subtopic: Private, Public, Protected;
Instance and Static Data Members

Objectives Covered:

4 Object-Oriented Programming

4.2 Declare, implement, and
access data members in classes

4.2.1 Private, public, protected

4.2.2 Instance data members

4.2.3 Static data members

Creating Constants and Encapsulation

Though static variables are generally set, they can be changed. There are times where you want to set a data member of a class to be a constant, however, and that can be done through using the final keyword. Encapsulation is the hiding of information. Making a data member private is, in this project, the act of encapsulation. Another point to remember in Java is to use the `String.format` method to format a number with a decimal point.

Purpose:

Upon completing this project, you will be able to understand and utilize constants and encapsulation in Java code.

Steps for Completion:

1. Open the project file entitled **Tea4.java**
2. Run the code as is. The static final keyword is used on line 42, and the message in the output terminal should refer to the two static prices referenced on lines 41 and 42.
3. Notice the private data member on line 10. Making this private encapsulates this data member within this class, which means you cannot use this data member in another class.
4. On line 27, format the sale price by typing:
System.out.println(t4.salePrice);
5. Save your changes and run the code. The sale price should display as 6.99 in the output. Close the file.

Project Details

Project File:

Tea4.java

Estimated Completion Time:

5 minutes

Video Reference:

Domain 4

Topic: Class Data Members

Subtopic: Use Static Final to Create Constants; Encapsulation

Objectives Covered:

4 Object-Oriented Programming

4.2 Declare, implement, and access data members in classes

4.2.4 Use static final to create constants

4.2.5 Describe encapsulation

Access Methods

Methods perform actions, often in the form of calculations. In this project, there are three methods you should notice. First the concatName method, which adds a first and last name with a space in between, is a public method, meaning it can be called anywhere within the app. On line 24, there is a last name, first name combination with a comma in between as a protected method. This allows this method to be called from anywhere within this package, or this specific file. On line 28, there is a private method for getting the initials from a customer. This method can only be used within this class. The pattern is similar to the way public, protected, and private classes are declared and used. Remember that a typical method uses parameters.

Purpose:

Upon completing this project, you will be able to understand and utilize access methods in Java.

Steps for Completion:

1. Open the project file entitled **ex431.java**
2. Replace the comment on line 21 with code that will return the first name, a space, and a last name.
3. Replace the comment on line 25 with code that will return the last name, a comma, a space, and the first name.
4. Replace the comment on line 29 with code that will return the first initials of the first name and last name with no space in between.
5. Save your changes.
6. Run the code. You should see the following output:
Pat Thomas
Thomas, Pat
PT
7. Open the project file entitled **ex432.java**
8. Replace the comments on lines 7 and 8 with calls to the orderTotal class, using the values specified in the comment.
9. Save your changes and run the code. Two orders should appear in the output with the price totals of 52.95 and 38.12.

Project Details

Project Files:

ex431.java, ex432.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 4

Topic: Methods

Subtopic: Private, Public, Protected Methods; Method Parameters

Objectives Covered:

- 4 Object-Oriented Programming
- 4.3 Declare, implement, and access data members in classes
 - 4.3.1 Private, public, protected
 - 4.3.2 Method parameters

Types and Methods

It is important to understand return type, void, and return value, which are all related topics. Whenever you are building a method, the first question to answer is, am I needing to return a value that I am going to use later? Whenever a method is going to return a value, you need two pieces in that method: the return type, which is a data type used right before the method name, and a return statement at the end of the method.

Another type of method you can use is a void, which runs and then it is done; it does not return a value. When it comes to using return values, one common mistake made is having the wrong data type specified either for the method itself, the variable used to return a value, or in one of the parameters used. Another mistake is forgetting the return statement altogether.

When comparing static and instance methods, static methods all belong to the class in a file and can be used anywhere within a class, whereas instance methods create instances of a class.

The overloading of methods is like the overloading of constructors. An overload is another method used in a Java file, which uses the same name as the method to overload, and the overload simply uses a different set of parameters.

Purpose:

Upon completing this project, you will be able to understand and utilize the return type, void, and return value, as well as instance methods, static methods, and overloading.

Steps for Completion:

1. Open the project file entitled **ex433.java**
2. Replace the comment on line 31 with a public static method declaration named **thankYou** that does not return any values.
3. Save your changes.
4. Run the code. You should see the following:
Your first order total is 52.95
Your second order total is 38.12
Thank you for your order.
5. To understand the difference between static and instance methods, notice how each method in this file is static, including the main method, order total, and void method. Each of these belongs to the class in this file and can be used anywhere within the class. Close the file.
6. Open the project file entitled **ex438.java**
7. Replace the comment on line 30 with a method declaration named **orderTotal** with the **qty**, **price**, and **location** variables as parameters.
8. Save your changes.
9. Run the code. You should see the following:
Your first order total is 52.95
Your second order total is 35.96.
10. Save and close all open files.

Project Details

Project Files:

ex433.java, ex438.java

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 4

Topic: Methods

Subtopic: Return Type and Value, Voids; Instance and Static Methods; Overloading

Objectives Covered:

- 4 Object-Oriented Programming
 - 4.3 Declare, implement, and access data members in classes
 - 4.3.3 Return type
 - 4.3.4 Void
 - 4.3.5 Return value
 - 4.3.6 Instance methods
 - 4.3.7 Static methods
 - 4.3.8 Overloading

Instantiation and Initialization

Initializing a class is the building of a class. Instantiating a class involves creating a new instance of that class. Null values can be used in classes and objects that are instantiated from those classes.

Purpose:

Upon completing this project, you will be able to initialize and instantiate a class, as well as use null values.

Steps for Completion:

1. Open the project file entitled **ex441.java**
2. Below the existing product declaration and print out, add another product declaration, naming it prod2, with a productID of 2, typeValue of Coffee, productNameValue of Colombian, and a price of 7.99.
3. Below the new declaration, add a print statement to print out the product2 product name. Make the message similar to the one on line 7, keeping in mind you are printing out a coffee product, not a tea product.
4. Save your changes.
5. Run the code. Your output should look similar to this:

Our first tea is Green Tea.

Our first coffee is Colombian.

6. Open the project file entitled **ex443.java**
7. Replace the comment on line 18 with a variable declaration for a variable named bagSize with a null default value.
8. Save your changes.
9. Run the code. The output should look like this:

The Green Tea Tea sells for 8.99

The Colombian Coffee sells for 7.99 per 11 oz. bag.

Project Details

Project Files:

ex441.java, ex443.java

Estimated Completion

Time:

5-10 minutes

Video Reference:

Domain 4

Topic: Instantiate and Use a Class Object

Subtopic: Instantiation and Initialization; Null Use

Objectives Covered:

4 Object-Oriented Programming

4.4 Instantiate and use class objects in programs

4.4.1 Instantiation

4.4.2 Initialization

4.4.3 Null

Accessing Data Members and Importing Classes

When it comes to accessing and modifying data members in a class, it is important to remember that you can set default values for those data members, but those values can be changed in the constructor used in the code.

You learned how to import packages in previous projects, but when it comes to importing a class, you do not need an import statement so long as the class is not private and is in the same package, meaning the same folder as the file you are trying to use it in. You will need to know the constructors for the class you are importing. If you do not remember, just start to instantiate an instance, and you will see the parameters that need to be filled in.

Purpose:

Upon completing this project, you will be able to access and modify data members, and import packages and classes.

Steps for Completion:

1. Open the project file entitled **TeaI.java**
2. Review the code as you will be using this class in another file.
3. Open the project file entitled **ex445.java**
4. Declare a new object based off the TeaI class. Name the object t6 and set the teaID to 6, the teaType to Regular, the teaName to Raspberry, and the teaPrice to 6.99.
5. Save your changes.
6. Run the code. You should see this message:
Our new tea is Raspberry.

Project Details

Project File:

TeaI.java, ex445.java

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 4

Topic: Instantiate and Use a Class Object

Subtopic: Access and Modify Data Members; Import Packages and Classes

Objectives Covered:

4 Object-Oriented Programming

4.4 Instantiate and use class objects in programs

4.4.4 Access and modify data members

4.4.7 Import packages and classes



Java

Domain 5

Fill-in-the-Blanks Domain 5

Instructions: While watching Domain 5, fill in the missing words according to the information presented by the instructor. [References where answers are found are in brackets.]

Syntax, Logic, and Runtime Errors

1. Debugging is the act of going through code and [] and [] problems with the code. [Print Statement Debugging]
2. Code can be debugged using javac, the Java [], and then the Java command. [Javac Command Output]
3. Logic errors can sometimes be the toughest errors to find, for the simple reason that they do not show up as [] errors while coding or errors in the console once code has been run. [Analyze for Logic Errors]
4. A red squiggly line is almost always an indication of a [] error. [Console Exceptions]
5. A stack trace is something seen when calling one or more [] creates an error. [Evaluate a Stack Trace]

Exception Handling

6. The catch block runs if there is any kind of [] within a try block. [Try Catch Finally]
7. An attempt to print an item outside of an array will result in an Array Index Out of [] exception. [Exception Class]
8. [] exceptions happen at runtime. [Exception Class Types]
9. Catching an exception requires a [] in order to call a built-in exception method. [Display Exception Information]

Debugging Code and Analyzing Logic Errors

Debugging is the act of going through code and finding and fixing problems with the code. One way to ensure that code is working properly is to use print statements when you suspect there may be a problem. Another way to debug code is to run the code from the command line interface using `javac`, the Java compiler, and then the `java` command. This is useful if you have data to input as part of running your code. However, there are things you must remember when running code from the command line. First, when compiling, make sure to include the `.java` extension. And, if the code takes arguments, remember to include those when running the `java` command.

Logic errors can be some of the toughest to find because they do not show up as syntax errors while coding, or as errors in the console once code has been run. When you have an if statement, the best way to test the code is to test each condition because it will help you discover potential logic errors. For your if statements with multiple conditions, such as the one in this project, make the most extreme condition, that is, the furthest from zero, your first condition.

Purpose:

Upon completing this project, you will be able to debug code using print statements and the `javac` command, as well as analyze code for logic errors.

Steps for Completion:

1. Open the project file entitled **ex511.java**
2. Run the code as it is to ensure that the print statements on lines 23 and 24 bring up the total amount of 49.95 and the sales tax amount of 2.997.
3. Fix the code so that the amounts of 49.95 and 52.95 (the amount plus the sales tax, not the sales tax by itself) print.
4. Save your changes.
5. Run the code. You should see amounts of 49.95 and 52.95. If necessary, format the second output to two decimals.
6. Open the project file entitled **ex512.java**
7. Click on the terminal tab and run the file through the compiler by typing:
javac ex512.java
8. Press the Enter key. After the new line in the terminal, test the parse code by typing:
javac ex512 15 3
9. Press the Enter key. The answer, 5, should display.
10. Open the project file entitled **ex513.java**
11. Run the code. The message should show that the customer status is Gold, but instead displays Silver.
12. Rearrange the code as needed so that the customer status will properly display as Gold for any orderQty of at least 80.
13. Save your changes.
14. Run the code again. The message should correctly show a customer status of Gold.
15. On line 5, change the order quantity by deleting 105, and typing: **50**
16. Save the file, and run the code. The message should show that the customer status is Silver.
17. Save and close your open files.

Project Details

Project Files:

ex511.java, ex512.java, ex513.java

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 5

Topic: Syntax, Logic, and Runtime Errors

Subtopics: Print Statement Debugging; Javac Command Output; Analyze for Logic Errors

Objectives Covered:

- 5 Code Compilation and Debugging
 - 5.1 Troubleshoot syntax errors, logic errors, and runtime errors
 - 5.1.1 Print statements
 - 5.1.2 Javac command output
 - 5.1.3 Logic errors

Console Exceptions and Stack Trace

When you are going through and attempting to fix code, sometimes you will catch errors before you run the code, and sometimes you will not see the errors until they come up in the console after you have run your code. Syntax errors are often signified by a red squiggly line underneath the code. Another form of error display is the use of a stack trace. This is something you will see when calling one or more methods creates an error. You will need to be able to interpret stack traces and know that the most recent method called is the one on top of the stack.

Purpose:

Upon completing this project, you will be able to fix exception errors and evaluate a stack trace.

Steps for Completion:

1. Open the project file entitled **ex514.java**
2. Type what is necessary to fix the syntax error on line 5.
3. Save the file.
4. Run the code. The code should partially work, but an `ArrayIndexOutOfBoundsException` exception should appear in the output.
5. Comment out the line of code causing the exception.
6. Save your changes and run the code again. You should see this output:

Dark Chocolate

Regular

Dark Chocolate

null

Sugar free

7. Open the project file entitled **ex515.java**
8. Run the code as it is. What does the 4 represent in the stack trace?
9. What does the java:21 represent?
10. What does the java:13 represent?
11. Comment out the line of code causing the exception.
12. Save your changes and run the code again. The Regular item should print.

Project Details

Project Files:

ex514.java, ex515.java

Estimated Completion Time:

10-15 minutes

Video Reference:

Domain 5

Topic: Syntax, Logic, and Runtime Errors

Subtopics: Console Exceptions; Evaluate a Stack Trace

Objectives Covered:

- 5 Code Compilation and Debugging
 - 5.1 Troubleshoot syntax errors, logic errors, and runtime errors
 - 5.1.4 Console exceptions
 - 5.1.5 Stack trace evaluation

Try Catch Finally and Exceptions

As a programmer, you will want to handle errors without your program crashing, if possible. However, if the program does have to stop, an easy-to-read error message as to why the program stopped can be helpful. You can do this through wrapping code in a try-catch-finally block. In this project, the goal is to print two items from an array within the try block. The catch block will run if there is any kind of exception, as you will see in the parameter for the catch block. The `ex` represents a variable to store the exception. The finally block runs no matter what happens with the try and catch portions. Technically, you can have a try and a catch with no finally, or a try and a finally with no catch.

An exception is a type of error within code. A catch statement can catch any exception, or a catch statement can be made to catch a certain class of exception. However, exceptions will not catch syntax errors.

Purpose:

Upon completing this project, you will be able to understand and use try catch finally blocks and exceptions.

Steps for Completion:

1. Open the project file entitled **ex521.java**
2. Run the code as it is. The first item should display, as well as the catch and finally messages.
3. Change the print item on line 15 by deleting 4, and typing: **3**
4. Save your changes and run the code. Both items should appear in the output, as well as the finally message. No catch message should appear because there should no longer be any errors in the code.
5. Open the project file entitled **ex522.java**
6. Replace the comment with the declaration of a catch block that specifically targets the `ArrayOutOfBoundsException` exception.
7. Save your changes.
8. Run the code. You should see a message indicating that you tried to print something that does not exist in the array.

Project Details

Project Files:

ex521.java, ex522.java

Estimated Completion Time:

5-10 minutes

Video Reference:

Domain 5

Topic: Exception Handling

Subtopics: Try Catch Finally;
Exception Class

Objectives Covered:

5 Code Compilation and Debugging

5.2 Implement exception handling

5.2.1 Try, catch, finally

5.2.2 Exception class

Exception Class Types

There are two types of exception class types: checked and unchecked. The checked exceptions are checked at compile time (before runtime), while the unchecked exceptions happen at runtime. Whenever you try to catch an exception, you also need a variable, which most people label as `e` or `ex`. This allows for calling some built-in exception methods. There are three of these methods displayed in your project: `getClass`, which will display the exception class; `getMessage`, which will display the exception message; and `fillInStackTrace`, which will display stack trace information.

Purpose:

Upon completing this project, you will be able to understand exception class types and display exception information.

Steps for Completion:

1. Open the project file entitled **ex523a.java**
2. Replace the comment in the second catch statement with the name of the exception that will catch errors as code runs.
3. Save your changes.
4. Run the code. You should see the following:
Item number 0 is Regular
Item number 3 is Sugar free
You have a different problem.
Code complete.
5. Open the project file entitled **ex524.java**
6. Replace the comment with three print statements: one to display the exception class, one to display the exception message, and one to display the exception fill in stack trace.
7. Save your changes.
8. Run the code. You should see the following:
Item number 0 is Regular
class java.lang.ArrayIndexOutOfBoundsException
4
java.lang.ArrayIndexOutOfBoundsException: 4
Code complete.
9. Save and close all of your open files.

Project Details

Project Files:

ex523a.java, ex524.java

Estimated Completion Time:

10 minutes

Video Reference:

Domain 5

Topic: Exception Handling

Subtopics: Exception Class Types;
Display Exception Information

Objectives Covered:

5 Code Compilation and Debugging

5.2 Implement exception handling

5.2.3 Exception class types

5.2.4 Display exception information



Java

Appendix

Java Domain I Outline Introduction

Course Opener

How to Study for the Exam

How to Set Up Visual Studio Code

The Use of Main

First Java File

Signature of Main

Consume a Class Instance

Command-Line Arguments

Basic Input and Output

Print Statements

Import and Use the Scanner Class

Variable Scopes

Variables in Blocks

Variables in Methods

Variables in Classes

Java Domain 2 Outline Primitive Type Variables

Data Types and Initialization

Lost Precision

Primitives vs. Wrapper Object Types

String Manipulation

String Classes and Literals

String.format Methods

String Operators

Primitive Data Type to String Conversion

Immutable Nature of Strings

Initialization, Null

Arrays and ArrayLists

One-Dimensional Arrays

Two-Dimensional Arrays

Other Array Creation Methods

ArrayLists and Adding Elements

ArrayLists and Removing Elements

Parsing, Casting, and Conversion

Casting and Conversion

Parse Strings to Numbers

Arithmetic and Assignment Operators

Arithmetic Operators

Assignment Operators

Operator Precedence

Java Domain 3 Outline Branching Statements

- If Statements
- Else Statements
- Else If Statements
- Switch Statements
- Single-Line vs. Block Statements
- Nesting Branch Statements
- Logical Operators
- Relational Operators

Loops

- While Loop
- Do While Loop
- For Loop
- For Each Loop
- Break
- Continue
- Nesting Loops
- Logical and Relational Operators
- Unary Operators

Java Domain 4 Outline Classes

- Build a Class
- Constructors
- Constructor Overloads
- Classes per Java File
- The This Keyword
- Inheritance
- Overriding

Class Data Members

- Private, Public, Protected
- Instance and Static Data Members
- Use Static Final to Create Constants
- Encapsulation

Methods

- Private, Public, Protected Methods
- Method Parameters
- Return Type and Value, Voids
- Instance and Static Methods
- Overloading

Instantiate and Use a Class Object

- Instantiation and Initialization
- Null Use
- Access and Modify Data Members
- Import Packages and Classes

Java Domain 5 Outline Syntax, Logic, and Runtime Errors

- Print Statement Debugging
- Javac Command Output
- Analyze for Logic Errors
- Console Exceptions
- Evaluate a Stack Trace

Exception Handling

- Try Catch Finally
- Exception Class
- Exception Class Types
- Display Exception Information

Course End

- Course Recap
- Conclusion

